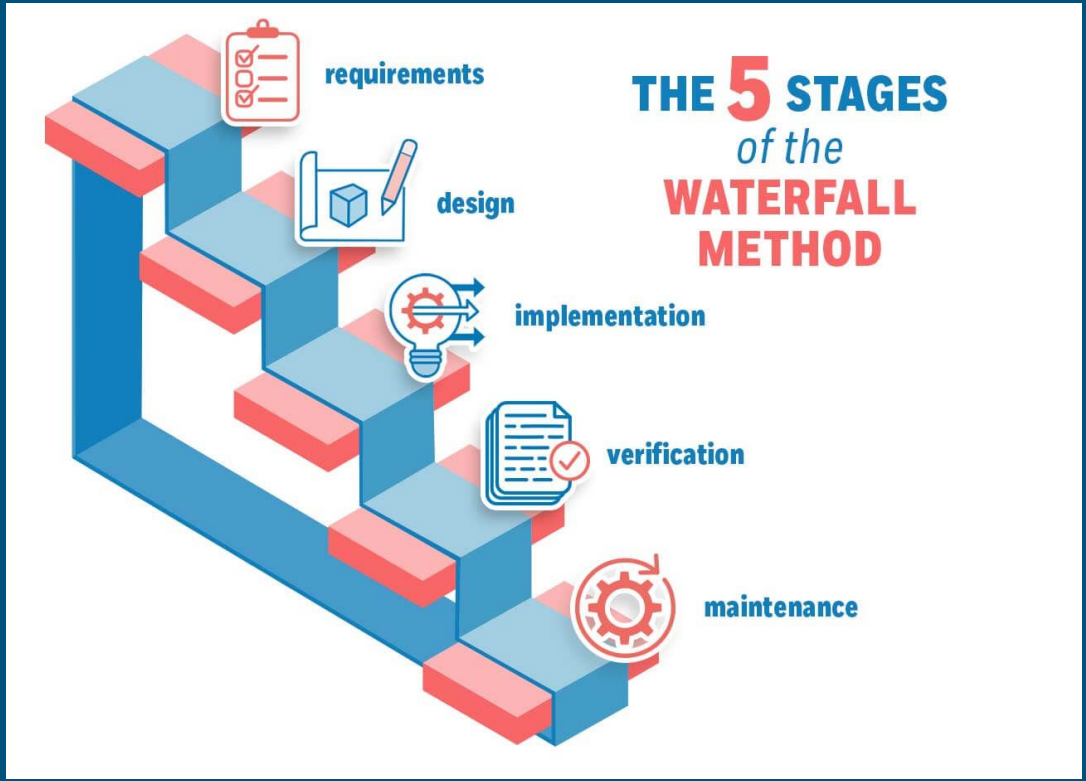
Lecture 4: Agile vs Waterfall

Software Engineering under Generative AI

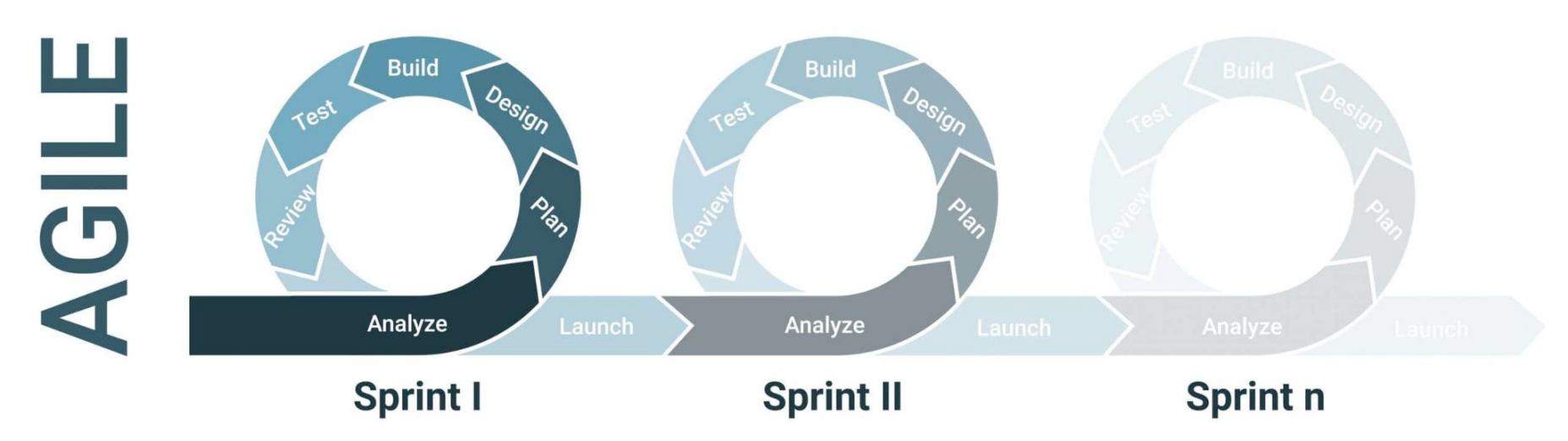
Madalina CROITORU

2026

Waterfall


THE 5 STAGES
of the
WATERFALL
METHOD
requirements
design
implementation
verification
maintenance

Agile


AGILE
Build
Design
Plan
Test
Review
Analyze
Launch
Sprint I
Build
Design
Plan
Test
Review
Analyze
Launch
Sprint II
Build
Design
Plan
Test
Review
Analyze
Launch
Sprint n

What happens when AI is in the loop?

- Blame assignment
- Responsability

Under different project management approaches - **does it make a difference**?

Can we learn from that difference to create **project management techniques** for hybrid AI - human teams?

Scenario for today : emergency response

Emergency response: ER

Emergency Response (ER) refers to the coordinated actions taken to manage critical situations such as fires, road accidents, medical emergencies, floods, or chemical leaks:

Begin with an alert from a caller, a witness, or an automated sensor, and require rapid interpretation of incomplete or uncertain information (i.e. “smoke near a school,” without knowing the exact address or the scale of the danger)

Responding effectively requires several aspects to be considered simultaneously:

- understanding what is happening,
- determining how urgent the situation is, deciding what resources are needed (such as ambulances, firefighters, or police),
- planning safe and efficient routes, and
- communicating clear instructions to responders and affected parties.

Because these elements depend on one another, mistakes or misunderstandings in early stages can lead to delays or inappropriate responses later, making coordination and consistency critical in emergency management

ER - components

- **Information understanding,** which includes extracting reliable facts from uncertain and often incomplete reports. Emergency calls or sensor messages may contain ambiguous locations, unclear incident types, or missing details about victims and hazards.
- **Decision and prioritisation**, where the system determines how urgent an incident is and what level of response is required. This step is critical because early misinterpretation or incorrect prioritisation can propagate through the rest of the system and lead to inappropriate or delayed responses.
- **Operational coordination**, which includes allocating resources, planning routes, and communicating with responders and external actors. Resource allocation must match available units to the needs of the incident
- **Route planning** must account for constraints such as traffic, accessibility, and safety conditions
- **Validation and consistency checking** are essential to detect contradictions between modules and prevent cascading failures.

Hybrid AI - human software

In this class you will consider a scenario in which some of the above modules are available by AI and some by human design (i.e. you).

The AI modules are of two types:

-sub-symbolic (i.e. black box AI with limited AI explanability)

-symbolic (i,.e. Based on a logic formalism, with the possibility of verifying formally the output)

AI components

AI Black Box — Incident Information Extraction

Takes raw caller text or description and outputs:

– Incident type (e.g., fire, accident, medical)
– Possible hazards
– Estimated number of victims
– Confidence score

Symbolic Component — Route Planning Engine

Takes location and resource information and outputs:

– Suggested route
– Estimated travel time
– Possible constraints (blocked roads, delays)

Human components

AI Output Interpretation Component
Interprets the classifier output, handles confidence values, and resolves uncertain or conflicting information.

Decision / Prioritisation Component
Assigns urgency level and response priority based on interpreted incident data.

Resource Allocation Component
Determines which responders, vehicles, or services are required.

Communication Component and Validation
Formats and sends structured messages to responders, hospitals, or authorities and checks for consistency

Waterfall vs Agile

Organise yourselves on 8 rows more or less balanced equally

- 4 rows - the front of the class - will do the waterfall model:
  - You will start with predefined module description sheets.
  - Each module is finalised before moving to integration with other modules.
  - You cannot modify the work done at a previous stage - you need to work around it
- 4 rows - the back of the class - will do the agile model:
  - Begin independently, allowed to exchange information earlier. Less time to produce a first version.

Waterfall groups - timing - 2h

Step 1 — Independent Design (30 min)
Work within individual groups only.
Each group designs its assigned component: define assumptions, choose input/output format, write pseudo-code. No discussion with other groups.

Step 2 — Same-Module Unification (45 min)
Discussion only with groups building the same module. The 4 groups responsible for the same component meet and merge their work into one shared module version. They must agree on one interface and one pseudo-code.

— First deadline at 1h15 — Submit unified module description.

Step 3 — Connected-Module Integration (45 min)
Discussion only with connected modules. Each unified module meets the modules it exchanges data with. They compare inputs and outputs and adjust formats once.

Short Integration Test included here.
Run a small scenario to check whether modules can exchange information correctly.

— Second deadline at 2h00 — Submit stabilized interfaces. TEST WATERFALL.

Step 4 — System Testing (30 min)
All modules interact. Run the full emergency scenario across the system. Record failures and trace where they originated.

— Third deadline at 2h30 — Submit final outputs and failure report.

Agile groups - timing - 2h

Iteration 1 — Initial Design (15 min)
Individual group work, but communication allowed. Groups create a first version of assumptions, input/output, and pseudo-code.

Iteration 2 — Same-Module Discussion (15 min)
Discussion with groups building the same module. Groups compare early designs and align major assumptions.

Iteration 3 — Connected-Module Discussion (30 min)
Discussion with connected modules. Groups compare expected inputs/outputs and revise their design.

Iteration 4 — Early Test (15 min)
Small scenario test. TEST Agile. Groups simulate partial execution and revise pseudo-code if problems appear.

— First deadline at 1h15 — Submit current module version.

Iteration 5 — Refinement Cycle (45 min)
Same-module and connected-module discussions allowed. Groups refine interfaces and logic. WATERFALL TEST.

— Second deadline at 2h00 — Submit stabilized module version.

Iteration 6 — Final Integration (30 min)
All modules interact. Run the full emergency scenario across the system and finalize outputs.

— Third deadline at 2h30 — Submit final outputs and failure report. TEST 2.